

StatComp Project 2: Scottish weather

Note:

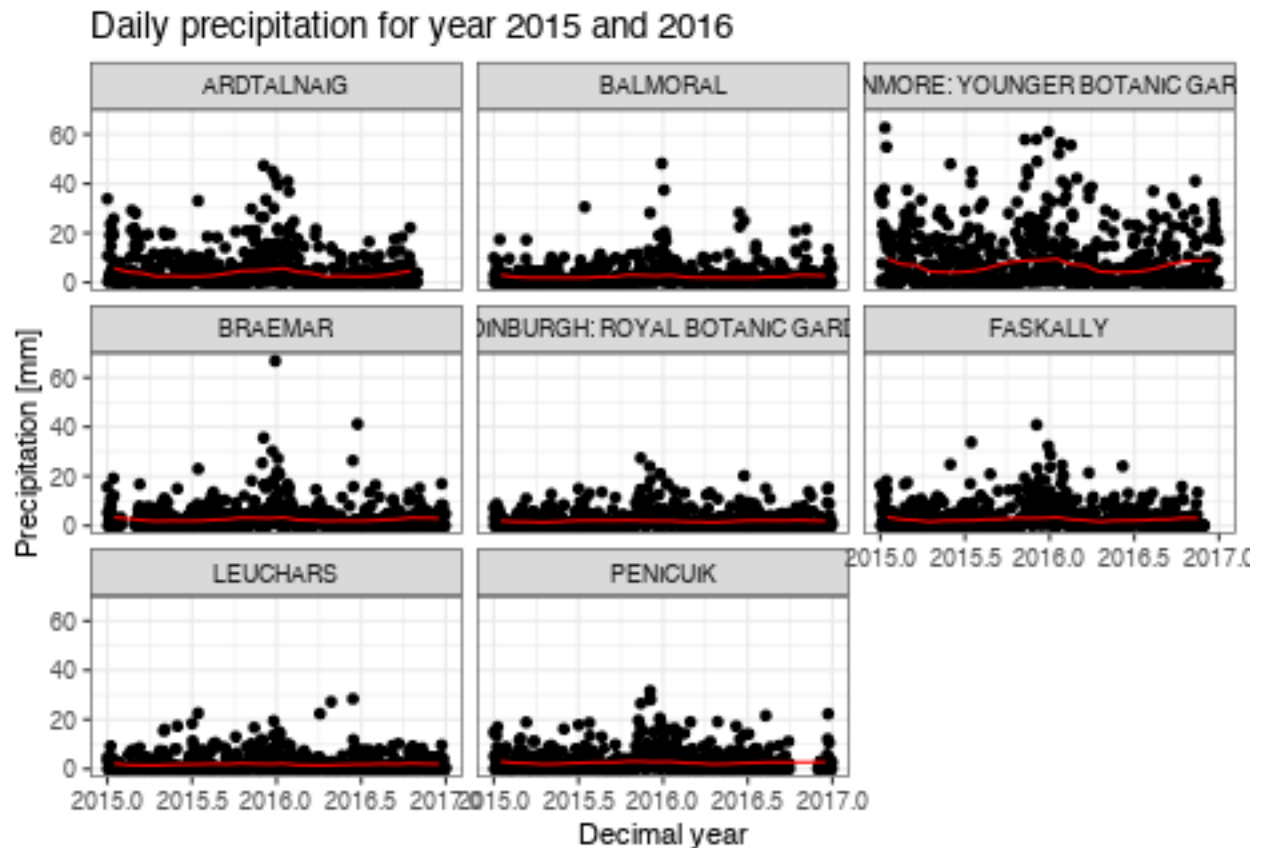
For both parts of this project, the student needs to organise their code to make reasonably efficient computations, and to organise the results so that they can be presented. There is not one single “best” solution; each part has multiple sensible approaches. Internal consistency and clarity of presentation is more important than particular model choices, etc, provided that the main questions are clearly addressed.

In particular for the spatial modelling part, this model solution includes more code and modelling alternatives than expected by the student solutions.

Seasonal variability

In order to analyse Scottish precipitation variability across different times of the year, i.e. seasonal variability, we start by loading the weather measurement data and combining it with station meta data. This is done in the `analysis.R` code. For illustration purposes, most of the analysis code operates on all the measured variables, but the results are only presented for precipitation, by filtering on ‘Element == “PRCP”’.

To get a feeling for the typical precipitation behaviour across Scotland, we plot the data for the years 2015 and 2016. We overlay a red curve for the monthly averages across all years.



Name	p_value	lwr	upr
ARDTALNAIG	0.000	0.0000000	0.0036821
BALMORAL	0.000	0.0000000	0.0036821
BENMORE: YOUNGER BOTANIC GARDE	0.000	0.0000000	0.0036821
BRAEMAR	0.000	0.0000000	0.0036821
EDINBURGH: ROYAL BOTANIC GARDE	0.671	0.6418789	0.7001211
FASKALLY	0.000	0.0000000	0.0036821
LEUCHARS	0.024	0.0145141	0.0334859
PENICUIK	0.000	0.0000000	0.0036821

It's clear that some stations, such as Ardtalnaig and the Benmore botanic gardens, have more overall variability than others, with an apparent seasonal pattern, but it's not obvious whether there is a consistent seasonal pattern for all locations.

The hypothesis we want to test is

H_0 :The precipitation distribution is the same in winter as in summer

H_1 :The winter and summer distributions have different expected values

We use the absolute value of basic expectation and probability estimates as test statistics, separately for each station:

$$T = |\text{winter average} - \text{summer average}|$$

We first compute the statistics T_0 on the original data. Since we don't have a model for the data distributions in winter or summer, we perform a randomised permutation test. First we add a **Summer** column to the data that is **TRUE** for data in April through September. We can then construct random permutations of this information within each station, and compute the corresponding test statistics, T_j , for $j = 1, \dots, N_T$. Under the null hypotheses the winter and summer observations are exchangeable, but not under the alternative hypotheses, so under the null hypotheses we would expect the original test statistics to be distributionally indistinguishable from the randomised test statistics, but under the alternative hypothesis we would expect the original test statistics to be larger, on average. We count the number of times $T_j \geq T_0$ for each of the two tests. The model for the counts and p-values is given as follows,

$$X = \sum_{j=1}^{N_T} \mathbb{I}(T_j \geq T_0) \sim \text{Bin}(N_T, p), \quad p = P(T_j \geq T_0 \mid H_0)$$

so that small values of p would mean that we reject the null hypotheses.

We don't have direct access to p , but can estimate them with $\hat{p} = x/N_T$, where x is the observed counts.

As shown in the Project2Hints document, when $N_T p(1-p)$ is large, an approximate 95 confidence interval for a p-value is provided by $\hat{p} \pm 2 \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{N_T}}$. However, it turns out that most of our observed counts are zero, so for those cases we instead use the alternative interval $[0, 1 - 0.025^{1/N_T}]$. We reject each H_0 if the corresponding p-value confidence interval is entirely below some significance threshold, but note that the true p-value might be larger than the threshold. In the `p_value_CI` function used to construct the intervals, I've also included the Bayesian credible interval method, which produces very similar results.

For $N_T = 1000$, the results are as follows:

We see that for all but two stations, the estimated p-values are zero, meaning that the stations appear to have clear seasonal patterns. For Edinburgh the p-value is surprisingly large, meaning that we cannot reject the null hypothesis of no seasonal differences. For Leuchars, the p-value estimate is positive, but the confidence interval still indicates a small value. This is the only interval for which the N_T value might need to be larger to be really confident; according to the approximate bounds in the Hints document we might need N_T as large as 10000 for a 0.02 interval width for *all* possible p-values, but a more refined reasoning, e.g. based on

Transform	Corr	Relative_SD
1: Original	0.8865708	0.2337959
2: Square root	0.6662696	0.1302978
3: Cube root	0.4148817	0.1080547

the p being smaller than 0.05, shows that $N_T = 1000$ is likely to be sufficient also for this case. The resulting p-value interval for Edinburgh is quite wide, but since the lower endpoint is very large, we don't need a more precise estimate to conclude that the true p-value is unlikely to be smaller than any reasonable statistical significance threshold. On my computer, $N_T = 1000$ took 51.244 seconds to run, when computing the tests for all three measurement variables. (When running only on precipitation data, $N_T = 10000$ took 3.5 minutes, which is consistent with having a third as much data, but analysing 10 times as many permutations test statistics.)

Spatial weather prediction

Note:

For this part of the project there were many possible routes to take. One option is to focus on a single model and discuss the results from that in detail. Another is to look at two models (e.g. without and with seasonal covariates) and discuss their similarities and differences. The solution presented here is much more extensive than required in terms of number of models,

Note:

For the option of including interaction effects between time and season (to check if the seasonal pattern changes over time), the solution here subtracts the average time before multiplying with the seasonal cos/sin covariates, to keep the main effect estimates relatively unaffected by the addition of the interactions. This also potentially avoids numerical singularity issues. However, the model predictions are unaffected by this technique; it's only the parameter estimates that become a bit easier to interpret, and the hints given on Piazza about this approach did not indicate a need to apply this improvement.

As noted in the Hints document for the project, the precipitation values are very skewed, with variance increasing with the mean value, and taking the square root of the monthly averages helps alleviate that issue, making a constant-variance model more plausible.

Note:

We can illustrate this by evaluating the correlations between the 12 monthly averages and the 12 corresponding standard deviations. In the illustration, we include the cube root, which might be an even better option, and has sometimes been used in practical applications. In addition to the correlation, we compute the standard deviation of the relative variability of the standard deviations. A smaller value means a more uniform standard deviation across the months.

```
vals <-
  ghcnd %>%
    filter(Element == "PRCP") %>%
    group_by(Year, Month) %>%
    summarise(Value = mean(Value), .groups = "drop")
rbind(cbind(vals, Transform = "1: Original"),
      cbind(vals %>% mutate(Value = sqrt(Value)), Transform = "2: Square root"),
      cbind(vals %>% mutate(Value = Value^(1/3)), Transform = "3: Cube root")) %>%
  group_by(Transform, Month) %>%
  summarise(Mean = mean(Value), SD = sd(Value), .groups = "drop") %>%
  group_by(Transform) %>%
  summarise(Corr = cor(Mean, SD), Relative_SD = sd(SD / mean(SD)), .groups = "drop") %>%
  knitr::kable() %>% kable_styling()
```

term	estimate	std.error	statistic	p.value
(Intercept)	2.4769477	1.3942775	1.776510	0.0757048
DecYear	0.0024109	0.0003968	6.076437	0.0000000
Latitude	-0.1359610	0.0201820	-6.736743	0.0000000
Longitude	-0.5464155	0.0107287	-50.930197	0.0000000
Elevation	0.0004022	0.0000769	5.230763	0.0000002

We see that the correlation drops from almost 90% for the original scale to around 2/3 for the square root, while the relative standard deviation drops from 0.23 to 0.13, which is a clear improvement.

Project solutions are not required to show this, and can use the square root idea from the Hints document without any special motivation beyond referring to the hint.

In the analysis code, we construct monthly data, `ghcnd_mon`, a set of model option definitions `models`, model formulas `models`, full parameter estimates `joint_fit`, associated predictions and scores `scores_plain`, and leave-station-out cross validation predictions and scores `scores_cross`.

Models and parameter estimates

Note:

The model estimates below are shown using `broom::tidy()`, but that wasn't widely mentioned in the course. `Plain summary()` output would be a fine alternative, that includes further information.

As a basic model for the square root of monthly average precipitation, we define

$$M_0 : \text{Value} \sim \text{Intercept} + \text{Longitude} + \text{Latitude} + \text{Elevation} + \text{DecYear}$$

By adding covariates $\cos(2\pi kt)$ and $\sin(2\pi kt)$ of frequency $k = 1, 2, \dots$ we can model also the seasonal variability, defining models M_1 , M_2 , M_3 , and M_4 , where the predictor expression for model M_K adds

$$\sum_{k=1}^K [\gamma_{c,k} \cos(2\pi kt) + \gamma_{s,k} \sin(2\pi kt)]$$

to the M_0 predictor expression, for model coefficients $\gamma_{c,k}$ and $\gamma_{s,k}$. The time variable t is defined to be `DecYear`, so that the lowest frequency $k = 1$ corresponds to a cosine function with a period on one full year.

Further to these models, we also consider models M_K^I , that add temporal-seasonal interaction terms of the lowest seasonal order only,

$$[\gamma_{c,1}^I(t - t_0) \cos(2\pi t) + \gamma_{s,1}^I(t - t_0) \sin(2\pi t)],$$

to allow the main part of the seasonal model to vary linearly in time. The offset time t_0 is chosen as the average of the observed time values, so that the interpretation of the other coefficients is preserved as much as feasible.

Note:

This offsetting by t_0 is also related to numerical stability concerns, but for the purposes of this project, that turns out not to be an issue, and the hints given on Piazza did not mention this option.

The parameter estimates from the non-seasonal M_0 model are as follows:

From the Longitude and Latitude parameter estimates, we see a clear decrease in precipitation when moving eastward, and a smaller decrease when moving northward. The effect of elevation and DecYear both appear to be very small, but both estimates are positive, showing a slight increase in precipitation over time, and with increasing elevation.

For the simplest seasonal model, M_1 , the estimates are very similar, with the addition of a clear seasonal effect:

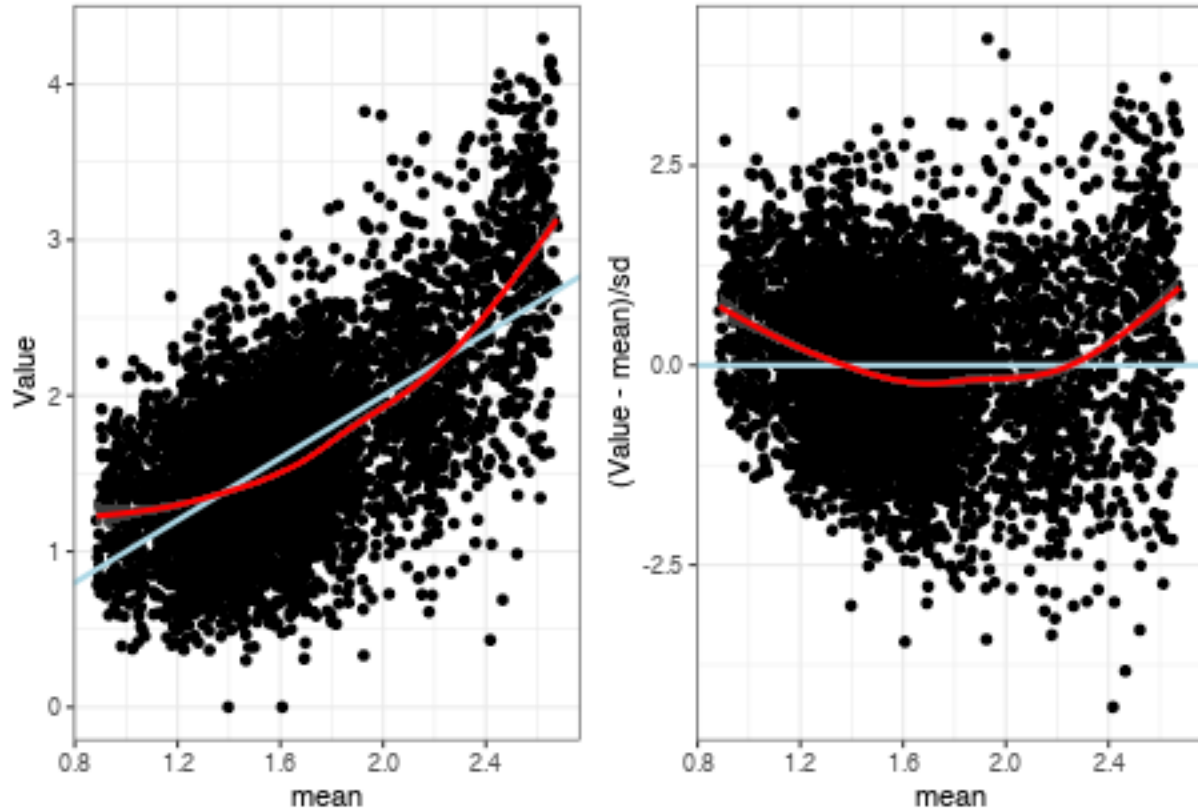
term	estimate	std.error	statistic	p.value
(Intercept)	2.4899466	1.3190655	1.887660	0.0591249
DecYear	0.0023757	0.0003754	6.328596	0.0000000
Latitude	-0.1349611	0.0190924	-7.068842	0.0000000
Longitude	-0.5466656	0.0101494	-53.861761	0.0000000
Elevation	0.0004040	0.0000727	5.554431	0.0000000
cos1	0.1828141	0.0089020	20.536227	0.0000000
sin1	-0.1320307	0.0088699	-14.885257	0.0000000

term	estimate	std.error	statistic	p.value
(Intercept)	2.4592323	1.3174027	1.8667279	0.0619934
DecYear	0.0023799	0.0003749	6.3479289	0.0000000
Latitude	-0.1345653	0.0190684	-7.0569664	0.0000000
Longitude	-0.5466930	0.0101365	-53.9333587	0.0000000
Elevation	0.0004039	0.0000726	5.5594641	0.0000000
cos1	0.1829453	0.0088907	20.5770652	0.0000000
sin1	-0.1319460	0.0088587	-14.8945919	0.0000000
I(cos1 * (DecYear - mean(DecYear)))	0.0021066	0.0005317	3.9617398	0.0000754
I(sin1 * (DecYear - mean(DecYear)))	0.0002409	0.0005284	0.4559262	0.6484613

When adding a time-season interaction effect, in model M_1^I , the sine term gives a very small contribution, but the cosine term has a magnitude similar to the main temporal DecYear effect, indicating a small long-term change in the seasonal pattern:

For a basic model fit assessment, we plot the M_1 model predictions against the observed values, as well as the standardised residuals. The identity line has been added in blue colour, and non-parametric smooths in red.

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
```



There is a pronounced non-linear pattern in the residuals, which indicates that there is some unmodelled behaviour in the data.

Model assessment

To more easily display differences between models, we will need the pairwise score differences in relation to model M_1 , which has one set of cos/sin functions as a seasonal model, and we add these reference scores as S_0 . We also rearrange the data to longer format, with a column `Score` to designate which score type (“SE” or “DS”) is stored in a new score value column, `S`.

```
scores <-
  scores_cross %>%
  pivot_longer(cols = c(SE, DS), names_to = "Score", values_to = "S")
scores <-
  left_join(
    scores,
    scores %>%
      filter(Model == "M1") %>%
      select(ID, Name, DecYear, Score, S0 = S),
    by = c("ID", "Name", "DecYear", "Score")
  )
```

By ordering the models according to decreasing (improving) Dawid-Sebastiani cross-validation scores, we see that the models mostly fall in order by number of parameters:

The main score improvement is between the non-seasonal model M_0 and the simplest seasonal model, M_1 . The seasonal models all have very similar average scores, but it's worth noting that the M_1^I model already has a lower score than the M_3 model, showing that even a basic temporally changing seasonal model improves

Model	order	interaction	S_DS	S_SE	dS_DS	dS_SE
M0	0	FALSE	-0.1814883	0.2842539	0.0820230	0.0213068
M1	1	FALSE	-0.2635113	0.2629471	0.0000000	0.0000000
M2	2	FALSE	-0.2642104	0.2625877	-0.0006991	-0.0003595
M3	3	FALSE	-0.2656085	0.2622205	-0.0020972	-0.0007266
M1I	1	TRUE	-0.2656647	0.2624740	-0.0021534	-0.0004731
M4	4	FALSE	-0.2659012	0.2621351	-0.0023899	-0.0008120
M2I	2	TRUE	-0.2663763	0.2621114	-0.0028650	-0.0008357
M3I	3	TRUE	-0.2677994	0.2617391	-0.0042881	-0.0012081
M4I	4	TRUE	-0.2681045	0.2616507	-0.0045932	-0.0012964

the model. However, since the score differences between the seasonal models are so small, the practical significance of this is unclear.

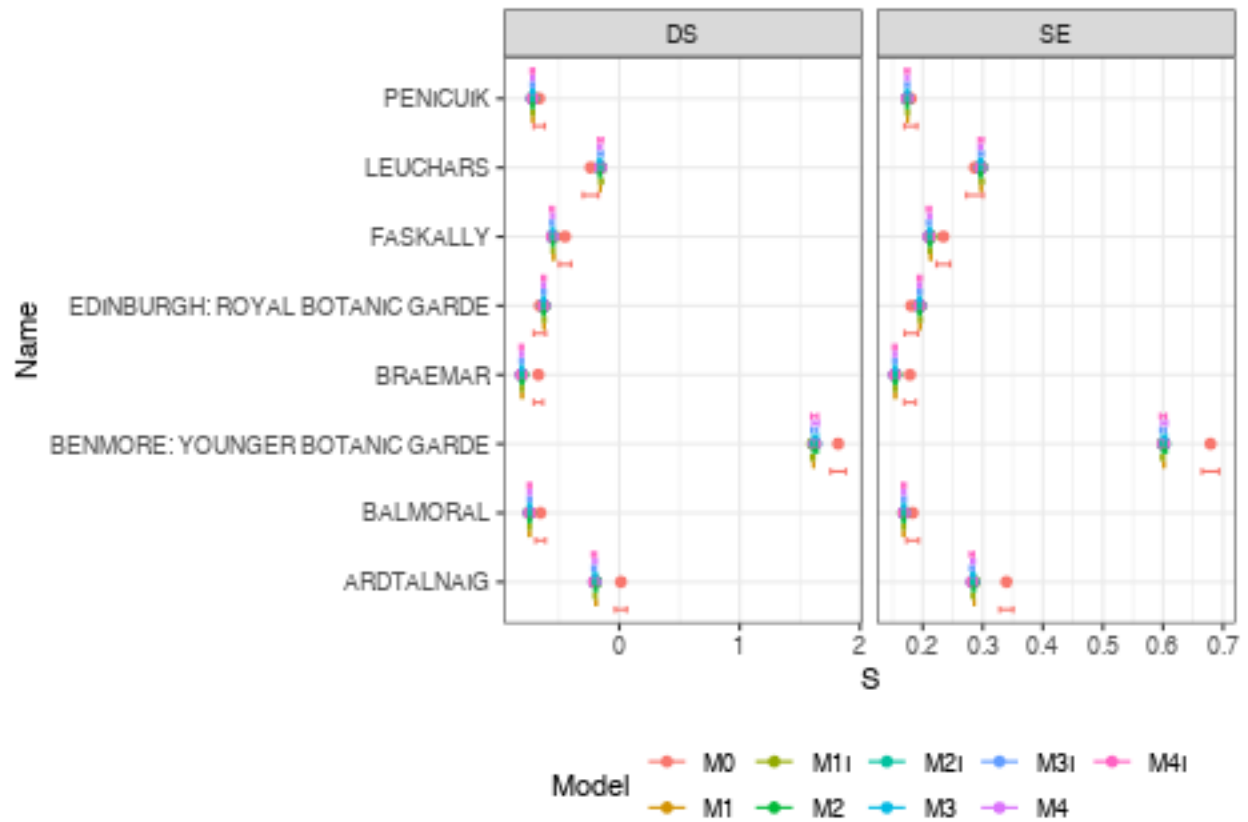
Station and season differences

Is the model equally good at predicting the different stations and across different times of the year?

We calculate the average scores (S) within each station, and the scores averaged within each month for each station. We also compute these averages with the reference model scores subtracted (dS), and the standard errors for the score averages (S_se) and score difference averages (dS_se).

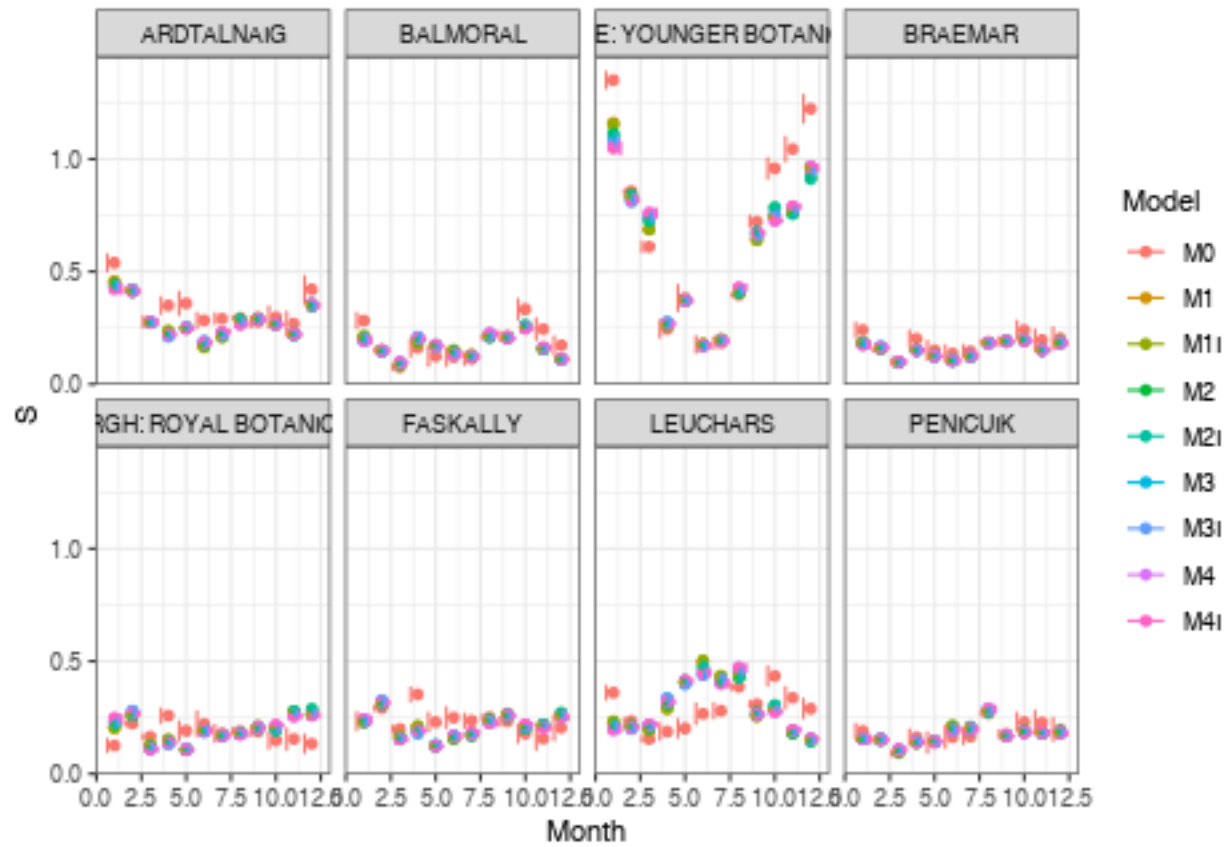
```
station_scores <-
  scores %>%
    group_by(ID, Name, order, interaction, Model, Score) %>%
    summarise(dS_se = sd(S - S0) / sqrt(n()),
              dS = mean(S - S0),
              S_se = sd(S) / sqrt(n()),
              S = mean(S),
              .groups = "drop")
month_scores <-
  scores %>%
    group_by(ID, Name, order, interaction, Model, Score, Month) %>%
    summarise(dS_se = sd(S - S0) / sqrt(n()),
              dS = mean(S - S0),
              S_se = sd(S) / sqrt(n()),
              S = mean(S),
              .groups = "drop")
```

In order to compare the prediction accuracy for the different stations, we plot the average cross-validation scores for each station. We also draw intervals based on $\pm$ two times the standard error for the average *difference* in scores relative to the reference model. This allows us to see if the model differences are statistically smaller or larger than the between-station differences:

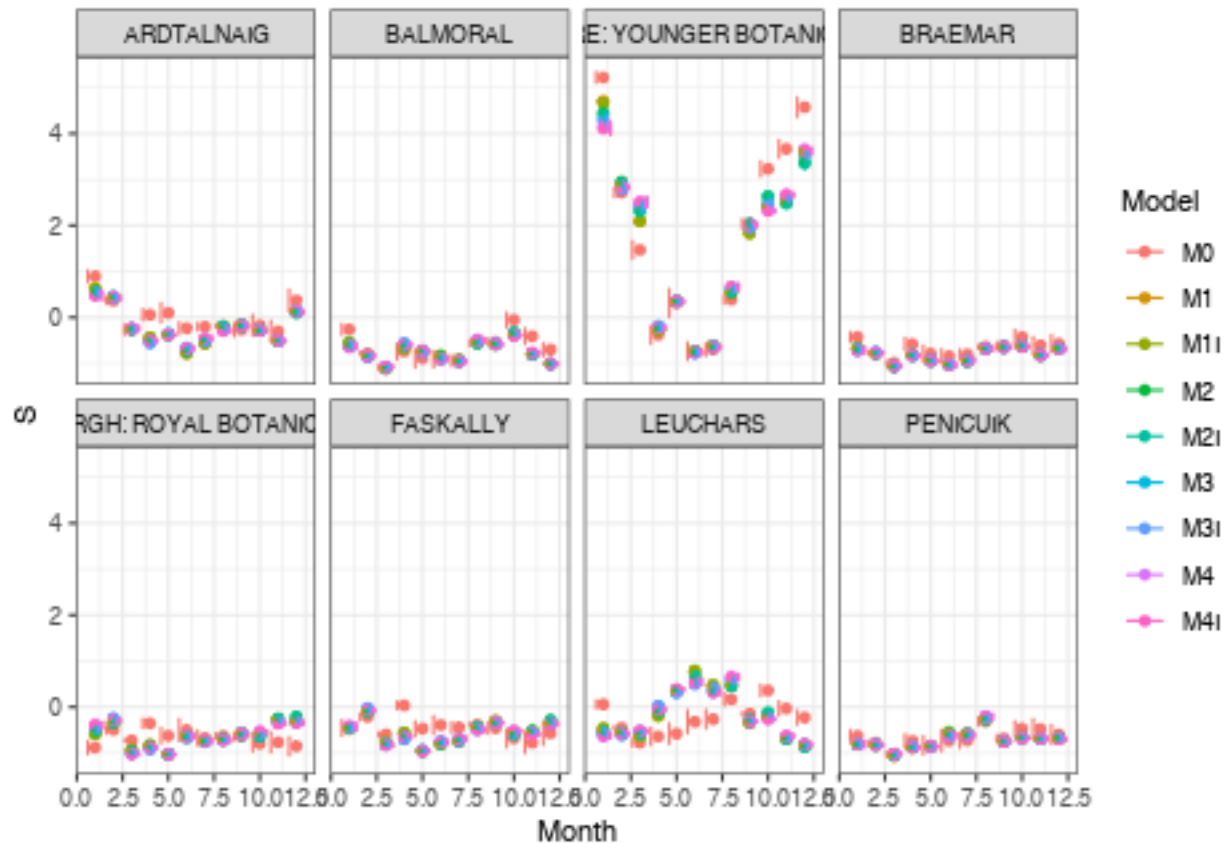


We see that the non-seasonal M0 model is consistently worse than all the other models, for all stations. But it's also clear that the between-station differences are much larger than the average model differences. This might be a symptom of the models not being able to capture nonlinear rainfall difference across space. We would need more advanced spatial models to better capture the true spatial behaviour.

Is the prediction accuracy the same across the whole year? Squared Error scores:



Dawid-Sebastiani scores:



For both scores, it's clear that the Benmore botanic garden station is challenging to model as part of basic models of the type we consider here. Looking at a map, one can find Benmore in the south-western parts of the highlands, near the west coast, whereas the other stations are further inland and/or to the east. In order to improve the model to work for a wider range of locations, one should consider a more flexible seasonal model that could be allowed to vary in amplitude across space in a non-linear manner.

Code appendix

Function definitions

```
# Place your function definitions that may be needed in report.Rmd
# in this file, including documentation.
# You can also include any needed library() calls here

suppressPackageStartupMessages(library(StatCompLab))
suppressPackageStartupMessages(library(tidyverse))

## Seasonal test statistics
##
## Compute permutation test statistics for equal vs non-equal distribution in
## summer and winter.
## Handles both original and randomised test statistics to avoid double grouping.
##
## @param data A data.frame including at least the variables `ID`, `Name`,
## `Element`, `Value`, and `Summer` (logical indicating data in summer months).
```

```

#' Note: In the project, we only use precipitation data, but this function was
#' written to allow the same analysis to be made for `TMAX` and `TMIN`.
#' @param random logical; if `TRUE`, randomise the `Summer` information
#' groupwise within (ID, Name, Element)
#'
#' @return A data.frame with `ID`, `Name`, `Element`,
#'   `TestA` (test statistics for equal/non-equal mean), and
#'   `TestB` (test statistics for equal/non-equal proportion of zero rain)

test_statistics <- function(data, random = FALSE) {
  data <- data %>%
    group_by(ID, Name, Element) %>%
    mutate(
      Summer =
        if (random) {
          sample(Summer, length(Summer), replace = FALSE)
        } else {
          Summer
        }
    ) %>%
    summarise(
      TestA = abs(mean(Value[!Summer]) - mean(Value[Summer])),
      TestB = abs(mean(Value[!Summer] > 0) - mean(Value[Summer] > 0)),
      .groups = "drop"
    )
}

#' p-value confidence interval
#'
#' To be called rowwise, so only a single p_value estimate is provided
#'
#' @param p_value A p-value estimate from a Binomial count proportion
#' @param N The number of trials used to compute the estimate
#' @param alpha The nominal interval error probability. Nominal coverage is
#'   `1 - alpha`.
#' @param bayesian logical, default `FALSE`; if `TRUE`, computes a Bayesian
#'   credible interval.
#' Code included here for demonstration, not used in the report.
#'
#' @return A data.frame with columns `lwr` and `upr`

p_value_CI <- function(p_value, N, alpha = 0.05, bayesian = FALSE) {
  if (bayesian) {
    data.frame(
      lwr = qbeta(alpha / 2,
                  1 + p_value * N,
                  1 + (1 - p_value) * N),
      upr = qbeta(1 - alpha / 2,
                  1 + p_value * N,
                  1 + (1 - p_value) * N)
    )
  } else {
    if (p_value == 0) {

```

```

    data.frame(lwr = 0,
               upr = 1 - (alpha/2)^(1/N))
  } else {
    se <- sqrt(p_value * (1-p_value) / N)
    data.frame(lwr = p_value - qnorm(1 - alpha/2) * se,
               upr = p_value - qnorm(alpha / 2) * se)
  }
}
}

#' Construct model formula
#'
#' @param order Non-negative integer indicating how many harmonic
#' orders to include in the seasonal model. Use `add_harmonics` to
#' add the required covariates to the data set.
#' @param extra extra model components, as a single string
#'
#' @return A formula for modelling `Value` as a function of `DecYear`,
#' `Latitude`, `Longitude`, `Elevation`, and optionally some seasonal harmonics.

make_formula <- function(order = 1, extra = NULL) {
  form <- "Value ~ 1 + DecYear + Latitude + Longitude + Elevation"
  if (order > 0) {
    form <- paste(c(form,
                    paste0("cos", seq_len(order),
                          " + sin", seq_len(order))
                    ),
                  collapse = " + ")
  }
  as.formula(paste0(c(form, extra), collapse = " + "))
}

#' Add harmonics to a data set
#'
#' @param data A data.frame with at least a `DecYear` column
#' @param order Non-negative integer indicating how many harmonic
#' orders to include in the seasonal model.
#'
#' @return A version of `data` with `Cos1`, `Sin1`, etc added, as
#' seasonal cycle basis functions of orders 1 through `order`, if `order` is
#' greater than zero.

add_harmonics <- function(data, order = 1) {
  for (k in seq_len(order)) {
    data[[paste0("cos", k)]] <- cos(2 * pi * data$DecYear * k)
    data[[paste0("sin", k)]] <- sin(2 * pi * data$DecYear * k)
  }
  data
}

```

```

#' In-sample predictions
#'
#' @param data Observation data.frame
#' @param formula A formula for `lm()`
#'
#' @return A data.frame with columns `mean` and `sd` with
#' model prediction mean and prediction standard deviation
#' for each observation.

plain_predictions <- function(data, form) {
  result <- data.frame(
    mean = numeric(nrow(data)),
    sd = numeric(nrow(data))
  )
  fit <- lm(form, data)
  pred <- predict(fit, newdata = data, se.fit = TRUE)
  result$mean <- pred$fit
  result$sd <- sqrt(pred$se.fit^2 + pred$residual.scale^2)
  result
}

#' Cross-validated predictions
#'
#' @param data Observation data.frame
#' @param formula A formula for `lm()`
#'
#' @return A data.frame with columns `mean` and `sd` with
#' model prediction mean and prediction standard deviation
#' for each observation, computed via leave-station-out estimation
#' and prediction for each station.

cross_validated_predictions <- function(data, form) {
  result <- data.frame(
    mean = numeric(nrow(data)),
    sd = numeric(nrow(data))
  )
  for (id in unique(data$ID)) {
    fit <- lm(form, data %>% filter(ID != id))
    pred <- predict(fit, newdata = data %>% filter(ID == id), se.fit = TRUE)
    result$mean[data$ID == id] <- pred$fit
    result$sd[data$ID == id] <- sqrt(pred$se.fit^2 + pred$residual.scale^2)
  }
  result
}

#' @param data A data.frame with (at least) a column `Value`
#' @param pred A data.frame with (at least) columns `mean`, and `sd`
#'
#' @return A data.frame with columns `SE` and `DS` with the squared error and
#' Dawid-Sebastiani scores for predicting `pred$Value` with prediction means
#' `pred$mean` and standard deviations `pred$sd`

calc_scores <- function(data, pred) {

```

```
data.frame(  
  SE = proper_score(  
    type = "se", obs = data$Value,  
    mean = pred$mean  
  ),  
  DS = proper_score(  
    type = "ds", obs = data$Value,  
    mean = pred$mean, sd = pred$sd  
  )  
)  
}
```